INTEGRAL DRAGON

MODULE 02 · WORKSHOP 04

Validation: Tests, Observability, Security

Confidence beats speed. Validation is what turns AI-assisted cleanup into engineering.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

Improvements only count if they are validated

AI-generated change + validation = trustworthy improvement

A PLAUSIBLE DIFF IS NOT A SAFE DIFF

- → **Validation turns cleanup into engineering** · tests, observability, and security each contribute a slice of confidence.
- → **Today is the guardrail workshop** · when we say no to fluent-but-unchecked AI output.

Coverage is a count. Confidence is a capability.

COVERAGE METER

92%

Looks great on a dashboard. Can pass with shallow assertions, mocked seams, and no real failure mode exercised.

CONFIDENCE METER

Signal

Tied to important behaviors. Fast feedback. Realistic fixtures. Failing tests are actionable and trusted.

→ **Risk-relevant tests** beat coverage percentages every time.

→ **Flaky tests erode trust** even when they raise the number.

Observability is part of maintainability

A system that cannot explain itself in operation is hard to change safely.

SIGNAL 01

Logs

Structured enough to diagnose. Correlation IDs and request context, not walls of free text.

SIGNAL 02

Metrics

Reflect real behavior and failure conditions. Not vanity uptime, not dashboards with no use case.

SIGNAL 03

Traces

Matter most when control flow crosses service or team boundaries. Stitch the request path together.

SIGNAL 04

Deployment visibility

What version is live, where, and is it behaving. Shortens the debug and verification loop.

Baseline security and supply-chain hygiene

- **Secrets management** · not casually embedded in code, docs, or chat history.
- **Dependency review** · visible at PR time, not a quarterly ritual.
- **Code scanning** · automated baseline, wired into CI rather than a one-off audit.
- **Vulnerability visibility** · the team knows what is outstanding and who owns it.
- **Safer defaults** · in code and configuration, so the easy path is the safe one.

Security is baseline engineering, not optional polish. AI can reduce this debt — and it can silently add to it.

A validation checklist for AI-assisted changes

- **What behavior changed?** Name it in one sentence without quoting the diff.
- **What tests prove it?** Which assertions would fail if the change were wrong?
- **What logs or metrics help verify it at runtime?** Can we see it after deploy?
- **What security or dependency risk changed?** New packages, new surfaces, new secrets.
- **What assumptions did the AI make?** Are they stated, and are they correct?

Re-use this verbatim as a PR template. The goal is to make evidence a pre-condition of merge.

Different changes need different evidence

DOCS

Accuracy review

Cross-check against the actual repo, not against a plausible-sounding README.

REFACTOR

Behavior checks

Regression tests on the boundaries that moved. Prove nothing user-visible changed.

OBSERVABILITY

Signal verification

Confirm logs, traces, and metrics actually appear, correlate, and carry useful context.

DEPENDENCY

Scan & review

Dependency-review, code scanning, license signals. A PR-time gate, not a quarterly audit.

MIGRATION

Rollback thinking

Forward plan, rollback plan, compatibility window. Schema changes are versioned.

Review an AI-generated change like an engineer

The task

- Read the AI-generated plan or diff end to end.
- Identify assumptions and unknowns the model glossed over.
- Check what validation evidence actually exists.
- Decide: approve, narrow, or reject — with rationale.

Debrief question

"What made this output look good at first, and what made it unreliable on inspection?"

Strong answers are specific about risk, aware of missing validation, and willing to cut scope rather than approve on vibe.

• CLOSE OUT

Coverage is a count.
Confidence is a capability.

Trust evidence, not fluency. Small validated improvements beat big speculative refactors — every time.